MPECSS: Scholtes Regularization with Adaptive Paths for Targeting B-stationary Points in MPECs

Saurabh (ORCID: 0009-0003-0458-8227)[a] and Kunwar Vijay Kumar Singh[b]

[a]Department of Mathematics, Atma Ram Sanatan Dharma College, University of Delhi, India

[b]Department of Mathematics, Atma Ram Sanatan Dharma College, University of Delhi, India

ARTICLE HISTORY

Compiled April 15, 2026

ABSTRACT

We propose MPECSS (MPEC Stationarity Search), an open-source algorithm for solving Mathematical Programs with Equilibrium Constraints (MPECs). Standard Scholtes regularization converges only to C-stationary points in general, a comparatively weak optimality concept providing no systematic mechanism for certifying the stronger B-stationary solutions required for genuine local optimality. MPECSS is implemented in Python via CasADi and IPOPT, with no commercial dependencies, fully open-source and reproducible. Convergence analysis establishes C-stationarity under standard assumptions; furthermore, MPECSS systematically certifies B-stationarity either directly when the multiplier sign test confirms S-stationarity under MPEC-LICQ or alternatively through LPEC (Linear Program with Equilibrium Constraints) verification. MPECSS achieves convergence rates of 96.3% [87.5% B-stationary among converged], 92.1% [85.4% B-stationary among converged] and 80% [85.4% B-stationary among converged] on MacMPEC, MPECLib and NOSBENCH respectively.

KEYWORDS

Complementarity constraints; Scholtes regularization; MPEC; Adaptive

CONTACT Saurabh. Email: 27098@arsd.du.ac.in
CONTACT K. V. K. Singh. Email: kunwar@du.ac.in

regularization; Open-source optimization

AMS CLASSIFICATION
90C33; 90C30; 65K05; 49M37; 90C90

1. Introduction

Mathematical Programs with Equilibrium Constraints (MPECs), equivalently referred to as Mathematical Programs with Complementarity Constraints (MPCCs).[1] Luo et al. [32] established the formal study of MPECs and formulated their foundational theory. In general, a standard MPEC can be formulated as follows [45]:

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \quad i = 1,\dots,m, \\ & h_j(x) = 0, \quad j = 1,\dots,p, \\ & G_\ell(x) \ge 0, \quad H_\ell(x) \ge 0, \quad \ell = 1,\dots,q, \\ & G_\ell(x)^\top H_\ell(x) = 0, \quad \ell = 1,\dots,q. \end{aligned} \tag{1}$$

Here, $f(x)$ is the objective function, $g_i(x)$ and $h_j(x)$ represent the standard inequality and equality constraints respectively.

The fundamental difficulty arising from complementarity constraints is that standard constraint qualifications such as Linear Independence Constraint Qualification (LICQ) and Mangasarian–Fromovitz Constraint Qualification (MFCQ) are violated at all feasible points [41]. Consequently, classical KKT condition is failed, and standard Nonlinear Programming (NLP) solvers applied directly to (1) will not find valid solutions. To overcome this computational barrier several specialized algorithms have been developed over the past two decades. Notable developments include regularization techniques [23, 31, 42, 43], sequential quadratic programming (SQP) approaches [16, 17], penalty methods [2, 20], and interior-point strategies [28, 38].

The most widely used approach for solving MPECs is *Scholtes regularization* [42],

[1] While the literature often uses MPEC and MPCC interchangeably, we adopt the term MPEC throughout this paper for consistency.

which replaces the complementarity product $G_\ell(x) \cdot H_\ell(x) = 0$ with the relaxed condition $G_\ell(x) \cdot H_\ell(x) \leq t_k$, and drives $t_k \to 0$ by solving a sequence of smooth NLP subproblems. While numerically robust and widely applicable, the standard formulation reduces t_k via a fixed geometric rule that is independent of solver progress, and is theoretically guaranteed to converge only to C-stationary points [19, 42], providing no built-in mechanism for targeting B-stationary. A detailed review of Scholtes regularization is given in Section 3.1; see also Hoheisel et al. [19] for a comprehensive theoretical and numerical comparison among different relaxation methods.

Existing methods face practical limitations: SQP approaches [16] often depend on commercial solvers like SNOPT [18]. The recent MPECopt algorithm of Nurkanović and Leyffer [35] is an open-source method with global convergence to B-stationary points and strong reported performance on the MacMPEC benchmark [29]; however, its current implementation depends on the commercial MATLAB environment and its strongest reported results use the commercial solver Gurobi.

Significant gaps remain in the practical MPEC software ecosystem. Many algorithms rely on commercial software (HSL, SNOPT, Gurobi), standard Scholtes regularization lacks built-in mechanisms for achieving stronger stationarity than C-stationarity. MPECSS is designed to close these implementation and reproducibility gaps.

MPECSS uses the Scholtes framework while adding an implementation-aware *adaptive regularization update* driven by the complementarity residual and an integrated *multiplier sign test* [41]. In the implementation, this sign test is used as an S-stationarity test on the G-multipliers and H-multipliers at numerically biactive indices; final B-stationarity certification is then performed in Phase III by a combination of MPEC-LICQ checks and, when needed, LPEC-based verification. The method is implemented entirely in Python using the open-source CasADi framework [1] and IPOPT [44], with no commercial dependencies.

The main contributions of this paper are such that *MPECSS algorithm:* A fully open-source, multi-phase algorithm for solving MPECs that combines Scholtes regularization with adaptive parameter control, a multiplier-based stationarity test, multistart mechanisms, and a Phase III certification pipeline combining LICQ checks, BNLP polishing, and LPEC refinement. *Adaptive regularization control:* A monitoring-

driven update of t_k based on the complementarity residual, stable biactive-set tracking, and stagnation counters, together with practical safeguards such as adaptive jumps, jump caps, and a floor on t_k. *Multiplier sign test:* A fast algebraic procedure that checks the converted G-multipliers and H-multipliers on numerically biactive indices to detect S-stationarity candidates and steer the subsequent certification logic. *Open-source benchmark infrastructure:* A reproducible benchmark runner for MacMPEC, MPECLib, and NOSBENCH, wide-schema CSV logging (300+ columns), environment capture, process-level isolation for per-problem timeout enforcement, resume support (via `--resume`), graceful recovery from timeout, interrupt, and out-of-memory events. The solver auto-selects between SQP+qpOASES (for problems with $n \leq 400$ when available) and IPOPT+MUMPS (otherwise).

2. Preliminaries

In this section we establish the notation and definitions used throughout the paper, recall the stationarity concepts and relevant constraint qualifications for MPECs.

2.1. Index sets and the MPEC Lagrangian

Throughout this section, and for the remainder of the paper, we assume that f, g_i $(i = 1, \ldots, m)$, h_j $(j = 1, \ldots, p)$, G_ℓ, and H_ℓ $(\ell = 1, \ldots, q)$ are twice continuously differentiable.

Consider the MPEC (1). We denote its feasible set by

$$\mathcal{F} = \big\{x \in \mathbb{R}^n : g_i(x) \leq 0,\ h_j(x) = 0,\ G_\ell(x) \geq 0,\ H_\ell(x) \geq 0,\ G_\ell(x)\, H_\ell(x) = 0,\ \forall\, i, j, \ell\big\}. \tag{2}$$

Let $x^* \in \mathcal{F}$ be a feasible point. Following standard MPEC literature [32, 41], we

partition the active complementarity components into the following index sets at x^*:

$$\mathcal{I}_g(x^*) = \{i \in \{1, \dots, m\} : g_i(x^*) = 0\}, \tag{3}$$

$$\mathcal{I}_{0+}(x^*) = \{\ell \in \{1, \dots, q\} : G_\ell(x^*) = 0,\ H_\ell(x^*) > 0\}, \tag{4}$$

$$\mathcal{I}_{+0}(x^*) = \{\ell \in \{1, \dots, q\} : G_\ell(x^*) > 0,\ H_\ell(x^*) = 0\}, \tag{5}$$

$$\mathcal{I}_{00}(x^*) = \{\ell \in \{1, \dots, q\} : G_\ell(x^*) = 0,\ H_\ell(x^*) = 0\}. \tag{6}$$

The set $\mathcal{I}_{0+}$ contains the indices where only G is active; the set $\mathcal{I}_{+0}$ contains the indices where only H is active; and the *biactive* (or *degenerate*) set $\mathcal{I}_{00}$ contains the indices where both complementarity functions equal zero simultaneously. We also write $\mathcal{I}_G = \mathcal{I}_{0+} \cup \mathcal{I}_{00}$ and $\mathcal{I}_H = \mathcal{I}_{+0} \cup \mathcal{I}_{00}$, and suppress the argument x^* universally when clear from context.

The *MPEC Lagrangian* associated with (1), which forms the basis for defining MPEC stationarity conditions [17, 41], is defined as

$$\mathcal{L}(x, \lambda, \mu, \gamma^G, \gamma^H) = f(x) + \sum_{i=1}^{m} \lambda_i\, g_i(x) + \sum_{j=1}^{p} \mu_j\, h_j(x) - \sum_{\ell=1}^{q} \gamma_\ell^G\, G_\ell(x) - \sum_{\ell=1}^{q} \gamma_\ell^H\, H_\ell(x), \tag{7}$$

where $\lambda \in \mathbb{R}^m$, $\mu \in \mathbb{R}^p$, and $\gamma^G, \gamma^H \in \mathbb{R}^q$ are the associated multiplier vectors.

2.2. Stationarity concepts

Proposition 2.1 (Failure of standard CQs [see e.g. 41])**.** *At any feasible point $x^* \in \mathcal{F}$ with $\mathcal{I}_{00}(x^*) \neq \emptyset$, both LICQ and MFCQ fail to hold for problem* (1)*. Consequently, the classical KKT condition does not provide a valid stationarity certificate at such points.*

At a feasible point x^* of (1), the complementarity constraints $G_\ell(x) \geq 0$, $H_\ell(x) \geq 0$, $G_\ell(x)^\top H_\ell(x) = 0$ together imply that standard LICQ and MFCQ necessarily fail (Proposition 2.1). MPEC-specific constraint qualifications are therefore introduced by considering the *tightened nonlinear program* (TNLP) [41], which replaces the comple-

mentarity constraints with the active-set equalities:

$$
\begin{aligned}
\mathrm{TNLP}(x^*): \quad & \min_{x\in\mathbb{R}^n} && f(x) \\
& \text{s.t.} && g_i(x) \le 0, \quad i = 1,\dots,m, \\
& && h_j(x) = 0, \quad j = 1,\dots,p, \\
& && G_\ell(x) = 0, \quad \ell \in \mathcal{I}_G, \\
& && H_\ell(x) = 0, \quad \ell \in \mathcal{I}_H.
\end{aligned} \tag{8}
$$

First we define stationary points, each of the following definitions, we assume that x^* is a feasible point of (1) and that there exist multipliers $(\lambda, \mu, \gamma^G, \gamma^H)$ satisfying the *basic stationarity system*:

$$
\nabla_x \mathcal{L}(x^*, \lambda, \mu, \gamma^G, \gamma^H) = 0, \tag{9}
$$

together with the standard sign and complementarity conditions

$$
\lambda_i \ge 0 \ \ (i \in \mathcal{I}_g), \quad \gamma_\ell^G = 0 \ \ (\ell \notin \mathcal{I}_G), \quad \gamma_\ell^H = 0 \ \ (\ell \notin \mathcal{I}_H).
$$

The vanishing of γ_ℓ^G for $\ell \notin \mathcal{I}_G$ and of γ_ℓ^H for $\ell \notin \mathcal{I}_H$ follows from complementary slackness.

Definition 2.2 (MPEC stationarity concepts [15, 32, 41, 45])**.** A feasible point x^* is called

- *W-stationary:* if there exist multipliers satisfying (9) with no further sign restrictions on $\gamma_\ell^G, \gamma_\ell^H$ for $\ell \in \mathcal{I}_{00}(x^*)$;
- *C-stationary:* if it is W-stationary and $\gamma_\ell^G \cdot \gamma_\ell^H \ge 0$ for all $\ell \in \mathcal{I}_{00}(x^*)$;
- *M-stationary:* if it is W-stationary and for each $\ell \in \mathcal{I}_{00}(x^*)$, either $(\gamma_\ell^G > 0, \gamma_\ell^H > 0)$ or $\gamma_\ell^G \cdot \gamma_\ell^H = 0$;
- *S-stationary:* if it is W-stationary and $\gamma_\ell^G \ge 0, \gamma_\ell^H \ge 0$ for all $\ell \in \mathcal{I}_{00}(x^*)$;
- *B-stationary:* if $\nabla f(x^*)^\top d \ge 0$ for all $d \in T_{\mathcal{F}}(x^*)$, where the *Bouligand tangent cone* is $T_{\mathcal{F}}(x^*) = \{d \in \mathbb{R}^n : \exists x^k \in \mathcal{F}, x^k \to x^*, t_k \downarrow 0 \text{ s.t. } \frac{x^k - x^*}{t_k} \to d\}$.

The strict hierarchy among multiplier-based notions is established in [15, 41, 45]:

$$\text{S-stat.} \Rightarrow \text{M-stat.} \Rightarrow \text{C-stat.} \Rightarrow \text{W-stat.} \tag{10}$$

Moreover, Scheel and Scholtes [41] show that S-stationarity implies B-stationarity without requiring constraint qualifications. Under MPEC-LICQ, the converse holds and the two are equivalent (B-stationary $\Leftrightarrow$ S-stationary).

By Definition 2.2, S-stationarity at x^* reduces to verifying the gradient condition (9) together with the sign conditions $\gamma_\ell^G \geq 0$ and $\gamma_\ell^H \geq 0$ for all $\ell \in \mathcal{I}_{00}(x^*)$. In practice, these conditions can be checked after any NLP solve by a fast algebraic test on the extracted multipliers—a procedure we call the *multiplier sign test*. In the implementation, a sign-test pass is treated as an *S-stationarity test*: it identifies a strong candidate point and steers the adaptive path, but final B-stationarity is reported only after the Phase III certification logic checks MPEC-LICQ or, if necessary, solves the LPEC verification problem. A complete description of how this test is integrated into the MPECSS algorithm is given in Section 3.

2.3. Constraint qualifications for MPECs

Definition 2.3 (MPEC-LICQ)**.** The *MPEC Linear Independence Constraint Qualification* [41] holds at a feasible point x^* if LICQ holds for the TNLP (8), i.e., the gradients

$$\big\{\nabla g_i(x^*)\big\}_{i\in\mathcal{I}_g},\ \ \big\{\nabla h_j(x^*)\big\}_{j=1}^{p},\ \ \big\{\nabla G_\ell(x^*)\big\}_{\ell\in\mathcal{I}_G},\ \ \big\{\nabla H_\ell(x^*)\big\}_{\ell\in\mathcal{I}_H}$$

are linearly independent.

Definition 2.4 (MPEC-MFCQ)**.** The *MPEC Mangasarian–Fromovitz Constraint Qualification* [41] holds at x^* if MFCQ holds for the TNLP (8). That is, the equality constraint gradients

$$\big\{\nabla h_j(x^*)\big\}_{j=1}^{p},\ \ \big\{\nabla G_\ell(x^*)\big\}_{\ell\in\mathcal{I}_G},\ \ \big\{\nabla H_\ell(x^*)\big\}_{\ell\in\mathcal{I}_H}$$

are linearly independent, and there exists a direction $d \in \mathbb{R}^n$ satisfying

$$\nabla g_i(x^*)^\top d < 0, \quad \forall\, i \in \mathcal{I}_g(x^*), \tag{11}$$

$$\nabla h_j(x^*)^\top d = 0, \quad \forall\, j = 1, \ldots, p, \tag{12}$$

$$\nabla G_\ell(x^*)^\top d = 0, \quad \forall\, \ell \in \mathcal{I}_G(x^*), \tag{13}$$

$$\nabla H_\ell(x^*)^\top d = 0, \quad \forall\, \ell \in \mathcal{I}_H(x^*). \tag{14}$$

MPEC-LICQ implies MPEC-MFCQ, but not vice versa. Under MPEC-LICQ, the multipliers $(\lambda, \mu, \gamma^G, \gamma^H)$ satisfying (9) are unique, while under MPEC-MFCQ they need not be.

With the notation, stationarity hierarchy, and constraint qualifications established above, Section 3 presents the MPECSS algorithm, which uses these concepts to construct an adaptive regularization path targeting B-stationary points of MPEC (1).

3. The Proposed Algorithm: MPECSS

In this section, we present the MPECSS algorithm. We first describe the main parts of the algorithm and then present a detailed step-by-step procedure.

3.1. Scholtes's regularization

As discussed in Section 1, the main source of difficulty in MPECs (1) is the complementarity conditions, which cause standard constraint qualifications to fail. In 2001, Scholtes [42] proposed a regularization technique, which relaxed the complementarity constraints $G_\ell(x) \cdot H_\ell(x) = 0$ to $G_\ell(x) \cdot H_\ell(x) \leq t_k$ for a sequence of positive parameters $t_k \to 0$ indexed by outer iteration k. This regularization transforms the original MPEC into a sequence of smooth NLP subproblems that can be solved using standard NLP solvers [16]. The key idea is that as t_k approaches zero, the solutions of the regularized problems approach a stationary point of the original MPEC.

After replacing the complementarity constraints with their regularized approximations, the regularized problem NLP(t_k) with parameter t_k at outer iteration k can be

formulated as [42]:

$$
\begin{aligned}
\mathrm{NLP}(t_k): \quad & \min_{x \in \mathbb{R}^n} && f(x) \\
& \text{s.t.} && g_i(x) \le 0, \quad i = 1, \dots, m, \\
& && h_j(x) = 0, \quad j = 1, \dots, p, \\
& && G_\ell(x) \ge 0, \quad \ell = 1, \dots, q, \\
& && H_\ell(x) \ge 0, \quad \ell = 1, \dots, q, \\
& && G_\ell(x) \cdot H_\ell(x) \le t_k, \quad \ell = 1, \dots, q.
\end{aligned} \tag{15}
$$

The sequence of solutions $\{\mathrm{NLP}(t_k)\}_{k=0}^{\infty}$ is expected to converge to a C-stationary point of the original MPEC as $t_k \to 0$ [42].

The regularization sequence $\{t_k\}$ is generated by a geometric reduction [19, 42]:

$$
t_{k+1} = \kappa \cdot t_k, \quad k = 0, 1, 2, \dots, \tag{16}
$$

where $t_0 > 0$ is the initial regularization parameter and $\kappa \in (0, 1)$ is the reduction factor. As $k \to \infty$, $t_k \to 0$, bringing the regularized problems progressively closer to the original MPEC (1).

Each subproblem $\mathrm{NLP}(t_{k+1})$ is initialized from the solution x^k of the previous subproblem $\mathrm{NLP}(t_k)$. This *warm-start* strategy means the solver follows a continuous *regularization path* $\{x^k\}$ as t_k decreases, rather than solving each subproblem from scratch [19, 39].

A solution x^k of $\mathrm{NLP}(t_k)$ is accepted when the KKT conditions are satisfied to a tolerance $\varepsilon_k > 0$. In practice, ε_k is reduced alongside t_k (e.g., $\varepsilon_k = O(t_k)$; see [19, 25]) to ensure that the accuracy of the solutions improves as t_k gets smaller.

Scholtes [42] proved that limit points of the regularized sequence are at best C-stationary for the MPEC. In general, C-stationarity is *strictly weaker* than B-stationary and the standard method provides no mechanism to detect or target B-stationary points [19, 42]. Alternative relaxation formulations have been proposed that guarantee stronger stationarity under weaker conditions (e.g., Kanzow and

Schwartz [24] achieve M-stationarity under MPEC-LICQ; DeMiguel et al. [7] propose a two-sided relaxation), but their theoretical stationarity guarantees can fail when NLP solvers return approximate solutions [25]. MPECSS takes a different approach: it preserves Scholtes's product relaxation which is numerically robust and fills the stationarity gap by combining the regularization path with an *adaptive path-following strategy* guided by the *multiplier sign test* (Section 2.2). At each outer iteration, the sign test checks the G-multipliers and H-multipliers of NLP(t_k) to confirm (or exclude) S-stationarity of the current iterate and uses this information to adjust $\{t_k\}$ accordingly. The full description of this adaptive mechanism is given in Sections 3.3–3.4. Notably, S-stationarity implies B-stationarity by Scheel and Scholtes [41], and under the MPEC-LICQ constraint qualification, S-stationarity and B-stationarity are equivalent. In the implementation, however, a sign-test pass is carried into Phase III, where B-stationarity is certified either by LICQ equivalence or by the LPEC check when LICQ is absent or fails to hold.

Note: MPECSS's code accepts benchmark models whose native complementarity format is not exactly the textbook form (15). For MPECLib and NOSBENCH, finite lower bounds on the complementarity functions are shifted into internal variables

$$\widehat{G}_\ell(x) := G_\ell(x) - \ell_\ell^G, \qquad \widehat{H}_\ell(x) := H_\ell(x) - \ell_\ell^H,$$

so that the solver always works with nonnegative internal pairs $\widehat{G}_\ell, \widehat{H}_\ell \geq 0$. For MacMPEC, the input handler also supports mixed and box-complementarity structures, including free G-components and upper-bound pairs of the form $(-G_\ell(x))(u_\ell - H_\ell(x)) \leq t_k$. The notation in the sequel is therefore to be understood as the solver's internal complementarity representation after this benchmark-specific normalization.

3.2. Phase I: Feasibility

The main MPECSS loop (Section 3.3) requires a starting point that approximately satisfies the complementarity conditions $G_\ell(x)^\top H_\ell(x) \approx 0$ with $G_\ell(x) \geq 0$ and $H_\ell(x) \geq 0$. A user-provided initial point x_0 may be far from complementarity-feasible (i.e., the products $G_\ell(x_0)^\top H_\ell(x_0)$ may be large). To address this, MPECSS begins with an op-

tional *feasibility phase* (Phase I), following standard MPEC feasibility preprocessing approaches [4, 35], that finds a good starting point satisfying the complementarity conditions. Before solving, the initial point x_0 is pushed into the strict interior of the variable bounds by a fraction $\nu = 0.1$, i.e., for each dimension i with finite bounds $[\ell_i, u_i]$, the starting value is restricted to $[\ell_i + \nu(u_i - \ell_i),\ u_i - \nu(u_i - \ell_i)]$.

Phase I then solves a sequence of up to three progressively more robust NLP formulations to find a complementarity-feasible point. The general structure minimizes $\Theta(x)$ of the complementarity conditions, subject to the original MPEC feasibility constraints:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \Theta(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \ldots, m, \\ & h_j(x) = 0, \quad j = 1, \ldots, p, \\ & G_\ell(x) \geq 0, \quad \ell = 1, \ldots, q, \\ & H_\ell(x) \geq 0, \quad \ell = 1, \ldots, q. \end{aligned} \tag{17}$$

The formulation of the objective $\Theta(x)$ is progressively adjusted in a cascade if the previous attempt fails to achieve a satisfactory complementarity residual ($r < 10^{-6}$). These attempts correspond to: (1) *L2 product minimisation* $\Theta(x) = \sum_{\ell=1}^{q} \big(G_\ell(x)H_\ell(x)\big)^2$ to standardly push products to zero [34]; (2) *Smooth L1 minimisation* $\Theta(x) = \sum_{\ell=1}^{q} \sqrt{\big(G_\ell(x)H_\ell(x)\big)^2 + \epsilon}$ (with $\epsilon = 10^{-10}$) to provide a less aggressive gradient when ill-conditioned [11, 14]; and (3) an *Epigraph formulation* which solves $\min_{x,\tau} \tau$ subject to the constraints in (17) as well as $\big(G_\ell(x)H_\ell(x)\big)^2 \leq \tau$ for all $\ell = 1, \ldots, q$ and $\tau \geq 0$, achieving stable gradients on severel infeasible problems [40]. For box-constrained complementarity pairs, the formulation aggregates both the lower residual $G_\ell(x)H_\ell(x)$ and, when a finite upper bound u_ℓ is present, the upper-bound residual $G_\ell(x)(u_\ell - H_\ell(x))$.

No regularization parameter appears in any of these formulations; each is a standard smooth NLP solved by IPOPT [44]. Upon convergence of any formulation, the resulting point $\bar{x}$ satisfies $G_\ell(\bar{x}) \cdot H_\ell(\bar{x}) \approx 0$ for all ℓ and lies in the feasible region of the original constraints. This point is then used as the warm-start x^0 for the main Scholtes outer loop. Algorithm 1 summarizes the practical Phase I procedure used to generate a

reliable warm start for the adaptive outer loop.

Algorithm 1 Phase I Feasibility

Require: Problem data P with complementarity functions and bounds, initial point x_0, solver options, maximum attempts $a_{\max}$, restart budget $r_{\max}$, seed s
Ensure: Warm-start point x^0 and Phase I metrics

```
n_x ← dim(x_0)
r_0 ← comp_res(x_0, P)
if n_x > 3000 then
    return x^0 ← x_0
end if
(ℓ_x, u_x) ← variable bounds from P
x_0 ← InteriorPush(x_0, ℓ_x, u_x, 0.1)
x_best ← x_0, r_best ← r_0
x_warm ← x_0, r_warm ← r_0
σ ← max(1, ‖x_0‖_2), D ← 50
Initialise residual and objective metrics
for a = 0, ..., a_max − 1 do
    Solve Phase I NLP regime a from x_0 if a = 0, else from x_warm
    Obtain candidate x̄ and residual r
    d ← ‖x̄ − x_0‖_2/σ
    if r < r_warm then
        x_warm ← x̄, r_warm ← r
    end if
    if r < r_best and (d < D or r < 10^−6) then
        x_best ← x̄, r_best ← r
    end if
    if r < 10^−6 then
        break
    end if
end for
if r_best > 10^−4 and r_max > 0 then
    Build deterministic restart candidates from lower bounds, upper bounds, midpoint, and Gaussian perturbations using a seed mixed with the problem name
    for j = 0, ..., min(r_max, |C|) − 1 do
        x_cand ← C_j
        for a = 0, 1 do
            Solve Phase I NLP regime a from x_cand if a = 0, else from x_best
            Obtain candidate x̄ and residual r
            d ← ‖x̄ − x_0‖_2/σ
            if r < r_best and (d < D or r < 10^−6) then
                x_best ← x̄, r_best ← r
            end if
            if r_best < 10^−4 then
                break
            end if
        end for
        if r_best < 10^−4 then
            break
        end if
    end for
end if
return x^0 ← x_best and associated metrics
```

Three baseline safety mechanisms enhance Phase I's reliability. First, a *structured multistart* samples deterministic lower-bound, upper-bound, and midpoint points together with Gaussian-perturbed starts whenever the best complementarity residual remains above 10^{-4}. Second, a *displacement guard* accepts a candidate only if its nor-

malized displacement from the initial point is below a fixed limit of 50, unless the residual is already below 10^{-6}. Third, the implementation records Phase I objective values for diagnostics, but the acceptance rule is driven by complementarity-residual improvement together with the displacement guard rather than by a separate original-objective filter.

In practice, the solver may fail to find an initial search direction from the user-provided starting point and report infeasibility immediately. To improve startup robustness without changing the theoretical Scholtes framework [42], MPECSS cycles through the three objective formulations described above. Each formulation offers a different objective geometry, and the cascade is arranged so that a more robust formulation is tried whenever the preceding one encounters numerical difficulty. For very large problems ($n > 3000$), Phase I is skipped entirely as it becomes computationally expensive; in this case, the user-provided starting point is passed directly to Phase II.

When Phase I achieves a complementarity residual $r_0 \leq 100\,\varepsilon_{\text{tol}}$, the implementation performs an early stationarity evaluation before entering the Phase II outer loop: it solves one NLP at a small regularization parameter $\bar{t} = \max(0.1\,r_0,\, 10^{-12})$ to extract multipliers and runs the sign test. Three outcomes are identified: If the sign test passes and $r_0 \leq \varepsilon_{\text{tol}}$, Phase II is skipped and the point proceeds directly to Phase III. If $r_0 \leq \varepsilon_{\text{tol}}$ but the sign test fails and the biactive count satisfies $|\mathcal{I}_{00}| \leq N_{\text{LPEC}}$ (default $N_{\text{LPEC}} = 15$), Phase III is executed directly since the LPEC enumeration ($2^{|\mathcal{I}_{00}|}$ branches) is computationally manageable. Otherwise, Phase II runs normally. Additionally, when Phase I produces $r_0 < t_0$, the initial regularization parameter is fast-forwarded to $t_0 \leftarrow \max(10\,r_0,\, \varepsilon_{\text{tol}}\,\tau)$ to preserve the refined starting point and avoid redundant outer iterations at large t values.

3.3. Phase II: Adaptive Path-Following Strategy with Sign Test

The main idea in MPECSS is to integrate a *multiplier sign test* into the regularization path-following framework to guide the sequence $\{t_k\}$ towards B-stationary points [41].

3.3.1. Solve regularized NLP subproblems

Phase I provides a robust warm start for the main loop. At each outer iteration k, MPECSS builds the smooth subproblem NLP(t_k) in CasADi and solves it with IPOPT [44] as its primary NLP solver. In the reported benchmark runs, IPOPT uses the open-source MUMPS linear solver [3] with tuned settings for robustness on ill-conditioned MPEC subproblems.[2] The codebase also provides an optional small-problem SQP+qpOASES [12] for problems with $n \leq 400$ variables, when qpOASES is available. For larger problems, CasADi's MX symbolic type is used together with a reverse-mode-favoring automatic differentiation setup. Solver templates and Jacobian functions are managed through bounded LRU caches with automatic removal, which helps keep memory usage predictable during long benchmark runs. Algorithm 2 provides the structure framework of Phase II.

Algorithm 2 Phase II Main Adaptive Path-Following Loop

Require: Warm-start x^0, initial regularization t_0, reduction factor κ, tolerance ε_{tol}, maximum iterations $k_{\max}$
Ensure: Candidate point x^* for Phase III

```
Set k ← 0; initialise history buffers, stagnation counters, and best-point tracker best
while k < k_max do
    Solve the regularized NLP at t_k from warm-start x^k to obtain (x^{k+1}, λ^{k+1})
    Convert multipliers and compute the homotopy residual r_{k+1}
    Run the sign test on I_00(x^{k+1}) and update best if the candidate improves the incumbent
    if r_{k+1} ≤ ε_tol and sign test passes then
        Set x* ← x^{k+1} and break
    end if
    Update t_{k+1} via Algorithm 3
    k ← k + 1
end while
if no convergence then
    x* ← best
end if
return x*
```

When the primary solve fails (e.g., due to infeasibility detection or numerical errors), MPECSS uses a small sequence of IPOPT retry configurations, including a Mehrotra-style predictor-corrector variant, relaxed tolerances, and gradient-based scal-

[2] MUMPS is configured with increased workspace allocation (`mumps_mem_percent` = 500), Hungarian scaling (`mumps_scaling` = 77), and pivot tolerance `mumps_pivtol` = 10^{-6}. IPOPT uses gradient-based NLP scaling, up to 4 second-order correction steps (`max_soc` = 4), and 3 watchdog trial iterations.

ing; if these attempts fail, it can switch to a Fischer–Burmeister reformulation of the complementarity constraints [14, 26, 44].

Each subproblem NLP(t_{k+1}) is warm-started from the solution x^k of the previous subproblem NLP(t_k), including primal and dual variables.[3] This *warm-start* strategy means the solver follows a continuous *regularization path* $\{x^k\}$ as t_k decreases, rather than solving each subproblem from scratch, greatly reducing the total number of NLP iterations.

3.3.2. Constraint formulation and multiplier sign test

For implementation consistency, the regularized constraints are passed to the NLP solver in a fixed order:

$$g(x) = \begin{bmatrix} g_{\text{orig}}(x) \\ G(x) + \delta_k \\ H(x) + \delta_k \\ G(x) \circ H(x) - t_k \end{bmatrix}, \tag{18}$$

where the tag orig means "original" and $g_{\text{orig}}(x)$ collects the inequality constraints inherited from the original MPEC model (before adding regularization terms), $\circ$ denotes componentwise product, and $\delta_k \geq 0$ is an optional regularization shift.[4] Let λ^{cas} be the solver multiplier vector associated with (18), where the tag "cas" means multipliers returned in the CasADi/IPOPT solver format. Because CasADi returns non-positive multipliers at active lower bounds for the G-constraints and H-constraints, MPECSS converts these complementarity constraints to MPEC-oriented multipliers by negation:

$$\gamma^{G,k} = -\lambda_G^{\text{cas}}, \qquad \gamma^{H,k} = -\lambda_H^{\text{cas}}, \tag{19}$$

The multiplier of the relaxed product constraint is still extracted and stored for test purposes, but it is *not* part of the S-stationarity sign test. All sign tests in MPECSS

[3] IPOPT warm-start parameters: `warm_start_bound_push` = `warm_start_bound_frac` = `warm_start_slack_bound_frac` = `warm_start_slack_bound_push` = 10^{-9}.

[4] In all reported numerical experiments, $\delta_k = 0$; the shift is retained in the formulation for generality.

are therefore performed only on the converted multipliers in Equation (19).

At each outer iteration k, after solving NLP(t_k), we extract the converted multipliers $(\gamma^{G,k}, \gamma^{H,k})$ for the shifted G-constraints and H-constraints. For biactive indices $\ell \in \mathcal{I}_{00}(x^k)$, the implemented sign test checks

$$\gamma_\ell^{G,k} \geq -\tau, \qquad \gamma_\ell^{H,k} \geq -\tau,$$

with a tolerance chosen in the implementation as $\tau = 10^{-6}$ by default. The adaptive quantity $\sigma_k = \max\{10^{-8}, \sqrt{t_k}\}$ is used instead for detecting the numerically biactive set. If the sign test passes and the complementarity residual is sufficiently small, Phase II returns an S-stationarity candidate to Phase III, where final B-stationarity is certified by the implemented LICQ/LPEC logic.

3.3.3. Adaptive t-update strategy

In the standard Scholtes regularization, the regularization parameter is reduced by a fixed geometric rule $t_{k+1} = \kappa\, t_k$ [42] regardless of the solver's progress. MPECSS replaces this with an *adaptive update strategy* that monitors the complementarity residual

$$r_k \;=\; \max_{\ell=1,\dots,q} \big| G_\ell(x^k)\, H_\ell(x^k) \big| \tag{20}$$

at every outer iteration k and adjusts t_k accordingly. For stationarity screening only, the code also tracks the biactive residual

$$\eta_k \;=\; \max_{\ell=1,\dots,q} \min\big(|G_\ell(x^k)|,\ |H_\ell(x^k)|\big). \tag{21}$$

Similar to the improvement ratio used in trust-region methods [34], we define the relative improvement

$$\rho_k \;=\; \frac{r_{k-1} - r_k}{\max(r_{k-1},\ \epsilon_{\text{floor}})}, \tag{22}$$

where ϵ_{floor} is a small positive guard to prevent division by zero (using the smallest normal float in the implementation). A value $\rho_k \approx 1$ indicates that the complementarity residual was nearly eliminated in one step, while small positive ρ_k indicates stagnation. The algorithm monitors whether convergence is sustained and the biactive set has stabilised. Specifically, define the *pathing condition* as

$$\alpha_L \;\leq\; \rho_k \;\leq\; \alpha_U, \tag{23}$$

with $\alpha_L = 0.2$ and $\alpha_U = 5.0$, where ρ_k is the relative improvement (22). The solver is said to be in *pathing mode* at iteration k when (23) holds, the biactive count $|\mathcal{I}_{00}|$ has been constant over the last three iterates, and $k \geq 4$. Pathing mode is *sustained* when the pathing counter $c \geq 2$ (i.e., the condition has held for at least two consecutive iterations).

Path 1: When pathing mode is sustained and $\rho_k > 0.5$—indicating strong, steady convergence—the algorithm uses an extra-aggressive reduction:

$$t_{k+1} = \kappa^2\, t_k. \tag{24}$$

Path 2: When pathing mode is sustained and $\rho_k > 0.1$, the algorithm maintains momentum with a capped reduction:

$$t_{k+1} = \min(\kappa,\, 0.5)\, t_k. \tag{25}$$

The cap at 0.5 prevents overly aggressive reduction when κ is set close to 1, ensuring robustness across a wide range of user-supplied κ values.

Path 3: Stagnation is tracked by a counter s that increments whenever $\rho_k < 0.05$ and resets to zero otherwise. Two sub-paths handle escalating stagnation. When $s \geq 4$, an aggressive jump is executed and the counter is reset:

$$t_{k+1} = \kappa^2\, t_k, \qquad s \leftarrow 0 \qquad \text{(Path 3a)}. \tag{26}$$

With the default $\kappa = 0.5$ this performs an aggressive reduction ($t_{k+1} = 0.25t_k$), forcing the solver to move past the stagnation point. When $2 \leq s < 4$, a milder post-stagnation reduction is used:

$$t_{k+1} = \min(\kappa,\, 0.5)\, t_k \qquad \text{(Path 3b)}. \tag{27}$$

Path 4: In all other cases—pathing mode is not sustained and the stagnation count is below threshold—the algorithm falls back to the standard reduction:

$$t_{k+1} = \kappa\, t_k. \tag{28}$$

In addition to the core t-update, the implementation includes several practical safeguards for difficult problems. After Phase I, if the warm-start is already nearly complementarity-feasible, the code can bypass Phase II and proceed directly to Phase III; alternatively, it can fast-forward the initial t_k. During Phase II, every outer NLP solve is protected by a wall-clock guard, and solver failures trigger the retry configurations described in Section 3.3.1 before the current iterate is discarded.

The aggressive stagnation jump (Path 3a in Algorithm 3) is governed by a hard cap of $b = 500$ total jump triggers; exceeding this cap prevents unbounded adaptive cycling and the outer loop declares stagnation. Additionally, the implementation monitors an *outer stagnation guard*; if the complementarity residual changes by less than 0.1% over a sliding window of 25 iterations, the outer loop exits early. Algorithm 3 summarizes the implemented adaptive update logic.

Algorithm 3 Phase IIa Adaptive t-Update

Require: Current t_k, residuals (r_k, r_{k-1}), stagnation counter s, pathing counter c, biactive count history, factors $(\kappa, \alpha_L, \alpha_U)$, iteration index k, remaining jump budget b

Ensure: Next regularization parameter t_{k+1}, updated counters (s, c, b)

Compute relative improvement ρ_k from (22)
Update stagnation counter: **if** $\rho_k < 0.05$ **then** $s \leftarrow s + 1$ **else** $s \leftarrow 0$
Set *biactive_stable* $\leftarrow$ (last 3 iterates share the same $|\mathcal{I}_{00}|$)
Evaluate pathing condition (23): *cond* $\leftarrow$ ($k \geq 4$ **and** *biactive_stable* **and** $\alpha_L \leq \rho_k \leq \alpha_U$)
Update pathing counter: **if** *cond* **then** $c \leftarrow c + 1$ **else** $c \leftarrow \max(0, c - 1)$
Set *pathing* $\leftarrow$ ($c \geq 2$)
if *pathing* **and** $\rho_k > 0.5$ **then**
    ▷ Path 1: superlinear mode
    $t_{k+1} \leftarrow \kappa^2 t_k$
else if *pathing* **and** $\rho_k > 0.1$ **then**
    ▷ Path 2: fast geometric mode
    $t_{k+1} \leftarrow \min(\kappa, 0.5)\, t_k$
else if $s \geq 4$ **then**
    ▷ Path 3a: aggressive stagnation jump; reset $s \leftarrow 0$
    if $b > 0$ **then** $t_{k+1} \leftarrow \kappa^2 t_k$, $b \leftarrow b - 1$, $s \leftarrow 0$ **else** $t_{k+1} \leftarrow \kappa\, t_k$
else if $s \geq 2$ **then**
    ▷ Path 3b: post-stagnation fast
    $t_{k+1} \leftarrow \min(\kappa, 0.5)\, t_k$
else
    ▷ Path 4: conservative mode
    $t_{k+1} \leftarrow \kappa\, t_k$
end if
$t_{k+1} \leftarrow \max(10^{-14}, t_{k+1})$
return t_{k+1}

3.4. Phase III: B-Stationarity Certification

After Phase II terminates with a candidate x^*, MPECSS enters a certification and refinement stage inspired by MPECopt-style active-set refinement and directional certification [35]. In the core solver, this stage first performs the LICQ/LPEC logic on the best complementarity-feasible point. In the benchmark pipeline used for the numerical experiments, it is then followed by BNLP polishing, an LPEC refinement loop, and a final post-check for B-stationarity.

Algorithm 4 summarizes the reported certification and refinement sequence.

When Phase II exits with $\varepsilon_{\text{tol}} < r_{\text{best}} \leq 1000\, \varepsilon_{\text{tol}}$, the benchmark pipeline performs a final push before certification. Gentle t reduction solves NLP(t) at progressively

smaller t values (factors 0.5, 0.1, 0.01 of the current estimate), rejecting any solution whose iterate norm diverges by more than $10\times$ from the current point. BNLP polish tries the BNLP (17) at multiple active-set tolerances and accepts only if it improves r_{comp}. Partition flipping flips, for each biactive index, the active-set assignment $G_\ell = 0 \leftrightarrow H_\ell = 0$ and re-solves the BNLP to find a feasible partition. If all strategies fail but $r_{\text{best}} \leq 10\,\varepsilon_{\text{tol}}$, the point is accepted under a relaxed tolerance and classified as C-stationary. This three-strategy cascade recovers problems whose homotopy residual is only slightly above the target tolerance.

Algorithm 4 Phase III B-Stationarity Certification and Refinement

Require: Candidate point x^* from Phase II
Ensure: Final status and refined point

```
if x* is complementarity-feasible then
    Check MPEC-LICQ at x*
    if sign test passes and MPEC-LICQ holds then
        Certify x* as B-stationary by S/B equivalence
    else
        Run LPEC-based certification on x*
    end if
end if
Run BNLP polishing; accept only if complementarity remains small and the objective does not worsen materially
Run trust-region LPEC refinement and a final B-stationarity post-check
Return the final point and certification status
```

Direct certification under sign test plus MPEC-LICQ: If the final iterate passes the multiplier sign test and MPEC-LICQ holds at that point, then the implementation certifies B-stationarity by the equivalence between S- and B-stationarity under MPEC-LICQ.

BNLP polishing: Even a good solution can be numerically rough. We solve a final simplified problem (BNLP) by fixing an active-set partition. Given index sets $\mathcal{I}_1 = \{\ell : G_\ell(x^*) \approx 0\}$ and $\mathcal{I}_2 = \{\ell : H_\ell(x^*) \approx 0\}$, the BNLP subproblem is:

$$
\begin{aligned}
\text{BNLP}(x^*, \mathcal{I}_1, \mathcal{I}_2): \quad & \min_{x \in \mathbb{R}^n} \; f(x) \\
& \text{s.t.} \quad g(x) \leq 0, \; h(x) = 0, \\
& \qquad G_\ell(x) = 0 \; (\ell \in \mathcal{I}_1), \; H_\ell(x) \geq 0 \; (\ell \in \mathcal{I}_1), \\
& \qquad H_\ell(x) = 0 \; (\ell \in \mathcal{I}_2), \; G_\ell(x) \geq 0 \; (\ell \in \mathcal{I}_2).
\end{aligned} \tag{29}
$$

For biactive indices ($\ell \in \mathcal{I}_{00}$), either G_ℓ or H_ℓ must be assigned to the equality branch; the implementation first uses the primary partition and then tries limited single-flip alternatives. In the core solver, a polished point is accepted only if its complementarity residual remains within the configured tolerance and its objective value does not worsen by more than a small relative/absolute tolerance. In the benchmark harness, an additional post-check may still relabel near-feasible failures with $r_{\text{comp}} \leq 10\,\varepsilon_{\text{tol}}$ after the final B-stationarity check.

LPEC-based certification and refinement: If sign test plus LICQ do not already certify the point, we use the geometric LPEC check to determine whether a feasible descent direction exists [17, 32]. The implemented LP model includes the active original constraints, active complementarity-side equalities, bound-consistent trust-region limits $|d_i| \leq \rho$, and branch-dependent inequalities for both lower-bound and upper-bound biactive complementarity indices. For a point with $|\mathcal{I}_{00}|$ biactive pairs, the implementation enumerates all XOR branches (either the G_ℓ side or the H_ℓ side stays active per biactive index), capped at 2^{15} branches for tractability, and solves the resulting LPs with SciPy's `linprog(..., method='highs')` backend [22]. If the branch enumeration times out or hits the branch cap without finding a descent direction, the outcome is treated as `stationarity_unverifiable` rather than as a certificate. In benchmark mode, this certification is followed by a trust-region refinement loop with parameters $\rho_{\text{init}} = 10^{-2}$, $\gamma_L = 0.1$, $N_{\text{out}} = 30$, $N_{\text{in}} = 15$, $tol_B = 10^{-10}$, and $tol_{\text{comp}} = 10^{-8}$.

In the reported experiments, every problem is post-processed by the full benchmark pipeline: the Phase II output is passed through BNLP polishing, LPEC refinement, and a final B-stationarity post-check, even when the initial sign test is favorable. This benchmark harness is therefore slightly richer than a bare call to `run_mpecss()`, and the numerical section reports the output of that harness rather than the raw Phase II endpoint alone.

4. Convergence Analysis

This section establishes the convergence properties of the MPECSS algorithm. We prove that the sequence of iterates generated by Phases I and II converges to feasible

points satisfying progressively stronger stationarity conditions, with Phase II guided by the adaptive path-following strategy to target B-stationary points.

4.1. Assumptions and Convergence Framework

Throughout this section, we make the following standard assumptions, which are common in the MPEC literature [32, 41, 42]:

Assumption 4.1 (Smoothness and Boundedness)**.** The functions f, g_i $(i = 1, \ldots, m)$, h_j $(j = 1, \ldots, p)$, G_ℓ, and H_ℓ $(\ell = 1, \ldots, q)$ are twice continuously differentiable. The feasible set

$$\mathcal{F} = \{x \in \mathbb{R}^n : g_i(x) \leq 0, h_j(x) = 0, G_\ell(x) \geq 0, H_\ell(x) \geq 0, G_\ell(x)H_\ell(x) = 0\}$$

is non-empty and bounded, or at least contains the level set $\{x : f(x) \leq f(x_0)\}$ under control.

Assumption 4.2 (Constraint Qualification)**.** At the (unknown) solution x^*, MPEC-LICQ (Definition 2.3) or MPEC-MFCQ (Definition 2.4) holds. This ensures that the Lagrange multipliers in the stationarity system are well-defined (or unique under MPEC-LICQ).

Assumption 4.3 (NLP Solver Accuracy)**.** At each outer iteration k, the solution x^k of the subproblem NLP(t_k) satisfies: (1) the KKT conditions with tolerance $\varepsilon_k > 0$, and (2) the feasibility constraints are satisfied exactly or to machine precision. The tolerance ε_k is progressively reduced as t_k decreases (e.g., $\varepsilon_k = O(t_k)$).

Assumption 4.3 reflects the practical implementation where IPOPT [44] is employed as the default NLP engine. The released code does *not* implement a proportional inner-tolerance schedule of the form $\varepsilon_k = O(t_k)$; instead, the benchmark configuration uses a fixed inner NLP tolerance inside each subproblem solve, together with the outer stopping tests and post-processing logic described in Section 3.4. The analysis below therefore relies only on the standard requirement that the regularized NLPs are solved to sufficiently accurate approximate KKT points.

4.2. Convergence of the Regularization Sequence

The foundation of MPECSS is Scholtes's regularization scheme [42], which has been extensively studied in the literature [19, 39, 43]. The following theorem, originally due to Scholtes [42], establishes that limit points of the regularized sequence are stationary for the original MPEC.

Theorem 4.4 (Scholtes's Regularization Convergence [42])**.** *Let $\{x^k\}$ be a sequence of local minimizers of the regularized subproblems NLP(t_k) with regularization parameters $t_k > 0$, $t_k \to 0$. If (i) the sequence $\{x^k\}$ is bounded, or has a convergent subsequence, and (ii) MPEC-LICQ (Definition 2.3) holds at the limit point x^*, then any limit point x^* of $\{x^k\}$ is feasible for the original MPEC* (1) *and is a C-stationary point (Definition 2.2).*

Proof. The key observation is that as $t_k \to 0$, the relaxed complementarity constraint $G_\ell(x)H_\ell(x) \leq t_k$ forces the complementarity condition $G_\ell(x)H_\ell(x) = 0$ in the limit. By continuity of the constraint functions and compactness of the feasible sets, the limit x^* satisfies MPEC-feasibility. C-stationarity follows from the KKT conditions of the subproblems and the sign constraints on the biactive multipliers imposed by active-set analysis. See Scholtes [42] and Scheel and Scholtes [41] for full details. □

Theorem 4.4 guarantees only C-stationarity, which is strictly weaker than B-stationarity in the hierarchy (10). Indeed, standard Scholtes provides no mechanism to enforce the sign positivity conditions required for S-stationarity. This is a fundamental limitation of naive geometric reduction.

Lemma 4.5 (Adaptive t Maintains Scholtes Guarantee)**.** *Under Assumptions 4.1–4.3, if the sequence $\{t_k\}$ generated by the implemented adaptive rule remains positive and $t_k \to 0$, then Theorem 4.4 applies and any limit point is C-stationary (or stronger if the subsequent certification stage succeeds).*

4.2.1. Convergence Toward S-Stationarity via Sign Test

The key innovation in MPECSS is to embed a *multiplier sign test* to steer the solution toward S-stationarity. At each outer iteration k, after solving NLP(t_k), the algorithm

extracts the converted multipliers $(\gamma^{G,k}, \gamma^{H,k})$ and checks the sign conditions at biactive indices $\ell \in \mathcal{I}_{00}(x^k)$:

$$\gamma_\ell^{G,k} \geq -\tau, \quad \gamma_\ell^{H,k} \geq -\tau, \tag{30}$$

with the fixed sign-test tolerance τ (default 10^{-6} in the released solver). If all biactive indices satisfy these conditions, the iterate x^k is accepted as an S-stationarity candidate. By [41], S-stationarity implies B-stationarity:

Theorem 4.6 (S-stationarity Implies B-stationarity [41])**.** *Let x^* be feasible for* (1)*. If x^* is S-stationary (Definition 2.2), then x^* is B-stationary (Definition 2.2). This implication holds without requiring any constraint qualification.*

In practice, the sign test is a fast algebraic check performed after each NLP solve. The biactive indices are detected adaptively with a tolerance $\sigma_k = \max(10^{-8}, \sqrt{t_k})$, ensuring that the screening set scales with the regularization parameter even though the sign threshold τ itself is fixed. A sign-test pass is therefore an implementation-level test for an S-stationarity candidate and for adaptive steering; final B-stationarity is reported only after the Phase III LICQ or LPEC checks.

4.3. Role of Feasibility Phase

4.3.1. Phase I (Feasibility) and Starting Points

Phase I solves the auxiliary problem (17):

$$\text{Phase I:} \quad \min_x \sum_{\ell=1}^{q} (G_\ell(x) H_\ell(x))^2 \quad \text{s.t.} \quad g_i(x) \leq 0, h_j(x) = 0, G_\ell(x) \geq 0, H_\ell(x) \geq 0. \tag{31}$$

This is a well-posed smooth NLP. Convergence of Phase I guarantees that the solution $\bar{x}$ satisfies the complementarity conditions approximately ($\sum_\ell (G_\ell(\bar{x}) H_\ell(\bar{x}))^2 \approx 0$) while respecting the original constraints. This warm-start enables Phase II to begin with a good feasible point, accelerating outer convergence.

Lemma 4.7 (Phase I Provides Feasible Warm-Start)**.** *Suppose IPOPT is applied to*

Phase I (31) *starting from* x_0*, and it converges to a point* $\bar{x}$ *satisfying KKT conditions to reasonable accuracy. Then* $\bar{x}$ *is approximately complementarity-feasible and lies in the feasible region of* (1)*. When Phase II is initialized with* $\bar{x}$ *and the configured regularization parameter* t_0 *(default* $t_0 = 1$ *in the released solver, with an additional fast-forward heuristic on near-feasible Phase I outputs), the regularized subproblems NLP*(t_k) *are well-posed from the outset.*

4.4. B-Stationarity Certification and LPEC Post-Solve

After Phase II converges (or reaches a maximum iteration count), MPECSS executes a *B-stationarity certification phase* (Phase III) to verify that the final iterate is B-stationary, especially when MPEC-LICQ may not hold.

4.4.1. MPEC-LICQ Check and Direct Certification

When the sign test is passed at the final iterate x^* and MPEC-LICQ holds at x^*, the algorithm certifies x^* as B-stationary by the equivalence between S- and B-stationarity under LICQ. If MPEC-LICQ does not hold or is uncertain, MPECSS solves an implementation-aware LPEC screen to provide a fall-back geometric verification:

Definition 4.8 (Implementation-Aware LPEC Screen [17, 32, 35])**.** Given a candidate x^*, the directional LP solved inside the implementation has the form

$$
\begin{aligned}
\mathrm{LP}(x^*, \rho, \beta): \quad & \min_{d \in \mathbb{R}^n} && \nabla f(x^*)^\top d \\
& \text{s.t.} && \nabla g_i(x^*)^\top d \le 0 \quad (i \in \mathcal{I}_g(x^*)), \\
& && \nabla h_j(x^*)^\top d = 0 \quad (j = 1, \dots, p), \\
& && \nabla G_\ell(x^*)^\top d = 0 \quad (\ell \in \mathcal{I}_G), \\
& && \nabla H_\ell(x^*)^\top d = 0 \quad (\ell \in \mathcal{I}_H), \\
& && \text{branch constraints determined by } \beta \text{ on each biactive index,} \\
& && -\rho \le d_i \le \rho, \qquad x^* + d \in [x_L, x_U].
\end{aligned}
\tag{32}
$$

Here β indexes the XOR branch choice for each biactive pair: either the G-side or the H-side remains active, with distinct inequality orientation for lower-bound and upper-bound box-complementarity cases. The implementation enumerates these branches up to a hard cap of 2^{15} and solves each LP with a finite wall-clock timeout. A completed enumeration with objective value ≥ 0 certifies that no descent direction was found within this implemented model; a timeout or cap-exceeded run yields an *unverifiable* outcome rather than a certificate.

Proposition 4.9 (LPEC Certification of B-Stationarity)**.** *If x^* is feasible for* (1)*, the full branch enumeration associated with* (32) *completes, and every branch LP has optimal value ≥ 0, then the implemented screen certifies x^* as B-stationary. Moreover, if MPEC-LICQ holds at x^*, B-stationarity is equivalent to S-stationarity (Definition 2.2). If the enumeration times out or exceeds the branch cap before completion, the implementation does not claim B-stationarity.*

4.4.2. Stationarity Hierarchy in MPECSS Outcomes

The MPECSS algorithm produces one of the following outcomes for a converged problem: *B-Stationary (sign test pass + MPEC-LICQ):* The solution passes the multiplier sign test and MPEC-LICQ is verified. The implementation then certifies B-stationarity by S/B equivalence under LICQ. *B-Stationary (LPEC certified):* Full LPEC certification (Proposition 4.9) confirms no descent direction exists, verifying B-stationarity. *S-Stationary candidate (sign test pass, MPEC-LICQ uncertain):* The multiplier sign test passes, confirming an S-stationarity candidate. When MPEC-LICQ holds at x^*, S-stationarity is equivalent to B-stationarity [41]; when MPEC-LICQ fails, LPEC certification (Proposition 4.9) is used to confirm or refute B-stationarity. *C-Stationary or unverifiable:* If the algorithm terminates with a converged complementarity-feasible point but without a B-stationarity certificate, the reported point is classified as C-stationary. If the LPEC screen times out, exceeds the branch cap, or the main solve fails, the implementation reports an unverifiable or non-converged status rather than claiming a theoretical stationarity level.

4.5. Convergence Summary and Computational Implications

Theorem 4.10 (Implementation-Aware Convergence Summary for MPECSS)**.** *Under Assumptions 4.1–4.3, the MPECSS algorithm (Phases I–III) generates a sequence* $\{x^k\}$ *such that: (i) The sequence is bounded (or has a limit point) in* $\mathbb{R}^n$*. (ii) Any limit point* x^* *is feasible for the original MPEC* (1)*. (iii) If the algorithm returns a complementarity-feasible solution with status "converged" and sign test PASS, then* x^* *is treated as an S-stationarity candidate under the implemented finite-tolerance test; the reported B-stationarity label is then produced by the Phase III LICQ/LPEC certification logic. (iv) If MPEC-LICQ (Definition 2.3) holds at* x^**, then S-stationarity and B-stationarity are equivalent. (v) If the full LPEC certification completes and passes, then* x^* *is certified as B-stationary. (vi) If the LPEC certification times out or the branch cap is reached without a completed proof and without a detected descent direction, the implementation reports* `stationarity_unverifiable` *rather than a B-stationarity certificate. (vii) The benchmark harness may additionally post-process some near-feasible non-converged outcomes with* $r_{\text{comp}} \leq 10\,\varepsilon_{\text{tol}}$*; this is an implementation-level acceptance rule and should not be read as a new asymptotic convergence theorem.*

While Theorem 4.10 establishes the implementation-aware asymptotic picture, the practical rate still depends strongly on problem conditioning, the chosen NLP solver, and the effectiveness of the adaptive t-update.

4.6. Local Optimality and Constraint Qualifications

It is important to note that the convergence results in this section apply to *feasible points* of the complementarity structure, not necessarily local minimizers of the original objective $f(x)$. Since B-stationarity is a necessary (but not sufficient) condition for local optimality [32, 41], MPECSS certifies that no descent direction exists within the feasible set (in the Bouligand sense). When combined with strict complementarity and higher-order analysis (not covered here), B-stationarity can be upgraded to local optimality; see [37] for details.

4.6.0.1. Remark on Unbounded Feasible Sets. If the feasible set is unbounded, Assumption 4.1 should be replaced with: the objective f has a lower bound on $\mathcal{F}$, or standard growth conditions hold. Under these modifications, the convergence analysis remains valid, and any limit point of an unbounded sequence $\{x^k\}$ satisfies the same stationarity certificates.

5. Numerical Experiments

This section reports the numerical performance of MPECSS across three major MPEC benchmark collections. Section 5.1 describes the experimental environment and algorithm configuration. Section 5.2 presents the complete MPECLib results. Sections 5.3 and 5.4 report the MacMPEC and NOSBENCH results respectively. Section 5.5 has the Dolan–Moré performance profile analysis [10] with MacMPEC benchmark suite.

5.1. Experimental Setup

The archived benchmark artifacts in this repository were not produced on a single manually curated machine image; instead, each benchmark run is accompanied by a run-environment JSON that records the command line, package versions, platform information, and hardware metadata for that particular run. The checked-in verified MPECLib run-environment file, for example, records Linux on an Intel Xeon platform with 31.35 GB RAM under Python 3.12.12, CasADi 3.7.2, and IPOPT [44] with the open-source linear solver MUMPS [3]. The code base also contains an SQP+qpOASES [12] route for small problems ($n \leq 400$). No commercial libraries (HSL, Gurobi, SNOPT) are required.

The benchmark-runner configuration used throughout all benchmarks is listed in Table 1. A per-problem wall-clock time limit of 3600 s is enforced via process-level isolation (using `multiprocessing.Process`); per-problem timeouts are enforced by monitoring the child process rather than POSIX signals, ensuring reliable enforcement even during C++ library (CasADi/IPOPT) execution.[5] Problems that exceed this

[5]To ensure single-threaded, deterministic execution, the environment variables `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `NUMEXPR_NUM_THREADS` are all set to 1. On Windows, process-level isolation via `multiprocessing.Process` with spawn context is used since signal-based timeouts cannot

limit are recorded as *timeout*. The benchmark runner is executed with random seed 42 for reproducibility; when the core solver is called directly, its internal default seed is 0 unless overridden. Every reported run uses the full benchmark pipeline: Phase I/II, BNLP polishing, LPEC refinement, and a final B-stationarity post-check.

Table 1. MPECSS default configuration used in all benchmark runs.

Parameter	Symbol	Value	Description
Initial regularization	t_0	1.0	Scholtes parameter at $k = 0$
Reduction factor	κ	0.5	Geometric base rate
Complementarity tolerance	ε_{tol}	10^{-6}	Convergence threshold
Stationarity tolerance	τ	10^{-6}	Sign-test tolerance
Maximum outer iterations	$k_{\max}$	3000	Phase II iteration cap
Floor stagnation window	—	20	Exit if $t_k = 10^{-14}$ unchanged
Adaptive update	—	enabled	product-residual and stagnation based t-control
Steering (sign test)	—	enabled	S-stationarity test plus Phase III certification
Wall-clock limit	$T_{\max}$	3600 s	Per-problem timeout
Benchmark seed	—	42	Benchmark-runner reproducibility

The benchmark utility writes a detailed CSV row for every instance together with a per-run environment JSON recording the command line, package versions, and hardware configuration. In addition to standard convergence fields, the logger records per-iteration test metrics (residuals, multiplier bounds, biactive counts, regime labels), phase-level timing splits, solver fallback flags, best-point tracking, and failure metadata. All reported times in this section are *wall-clock* times measured by the benchmark harness, not process CPU times. Unless stated otherwise, median times in the size-stratified tables are computed over converged instances only. The size-class labels reported below are taken directly from the machine-generated `problem_size_mode` field in the archived CSV files: *small* if $n_x < 50$, *medium* if $50 \leq n_x < 500$, and *large* if $n_x \geq 500$.

Each problem is assigned one of the following outcome classifications based on the solver's termination status and stationarity certificate: (i) *B-stationary*—the solver reports `converged` and the Phase III LICQ/LPEC verification certifies B-stationarity; (ii) *C-stationary*—the solver reports `converged` but only C-stationarity can be certified (B-stationarity verification fails or is inconclusive); (iii) *Timeout*—the 3600 s wall-clock limit is exceeded before convergence; (iv) *Other failures*—complementarity infeasibility (`comp_infeasible`), NLP solver failure (`nlp_failure`), out-of-memory (`oom`), worker crash, or model-loading failure. Most *converged* results satisfy the tar-

interrupt blocking IPOPT calls.

get threshold $r_{\text{comp}} \leq 10^{-6}$ directly; however, in the benchmark harness a final B-stationarity post-check may also relabel a small number of near-feasible outcomes with $r_{\text{comp}} \leq 10\,\varepsilon_{\text{tol}}$ after the final certification stage.

Three benchmark collections are used: MPECLib, MacMPEC and NOSBENCH.

5.2. MPECLib Benchmark Results

MPECLib [9] is a standard MPEC test set maintained by GAMS World originally containing 92 problems drawn from bilevel optimization, contact mechanics, game theory, and structural optimization. Of these, 3 problems (`three`, `bard2`, and `qvi`) are excluded from our experiments as they contain modeling constructs that fall outside the pure MPEC formulation (1)—specifically, these problems include disjunctive constraints, general complementarity systems, or other extensions not directly expressible as standard complementarity pairs. The archived raw CSV still logs these three instances with status `unsupported_model`; the analysis below filters them out and therefore reports the supported 89-problem subset. In particular, the archived CSV shows that `mss` is treated as a supported MPEC instance and converges to a B-stationary point. We solve these 89 problems with MPECSS using the configuration in Table 1.

5.2.1. Summary Statistics

Table 2 summarises the overall outcomes. MPECSS converged on 82 of 89 supported problems (92.1% [85.4% B-stationary among converged]), certifying B-stationarity on 70 instances (78.7%) and C-stationarity on the remaining 12 (13.5%). The remaining 7 problems (7.9%) did not converge: 4 reached the 3600 s timeout limit, 2 terminated with stationarity unverifiable, and 1 was classified as complementarity-infeasible.

MPECSS demonstrates strong performance across all problem size categories. All 29 small problems ($n_x < 50$) converge successfully (100%), with all 29 achieving B-stationarity. All 47 medium-scale instances ($50 \leq n_x < 500$) also converge successfully, with 36 certified as B-stationary and 11 as C-stationary. For the 13 large-scale problems ($n_x \geq 500$), 6 converge successfully (46.2%), with 5 achieving B-

Table 2. MPECSS outcome summary on the supported MPECLib benchmark (89 problems, 3600 s time limit per problem).

Outcome	Count	Percentage
Converged – B-stationary	70	78.7%
Converged – C-stationary	12	13.5%
Total converged	*82*	*92.1%*
Timeout (>3600 s)	4	4.5%
Stationarity unverifiable	2	2.2%
Complementarity infeasible	1	1.1%
Total	**89**	**100%**

stationarity and 1 reaching C-stationarity. The non-converging cases are concentrated in the `tinque` family (4 timeout instances, with the remaining 2 instances converging) and the `frictional_block` family (2 stationarity-unverifiable instances and 1 complementarity-infeasible termination). These large-scale problems with thousands of complementarity constraints pose significant computational challenges for the open-source MUMPS linear solver.

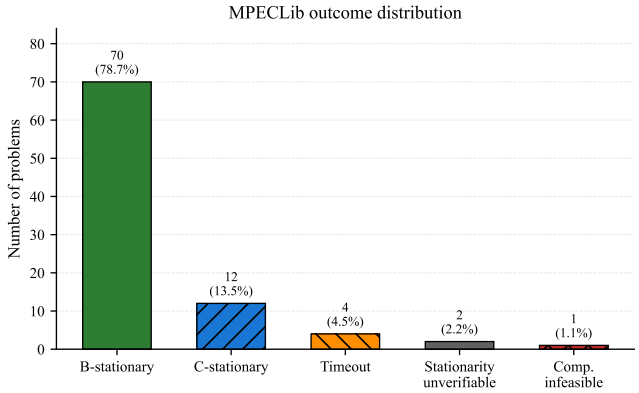

Figure 1. MPECSS outcome distribution on the supported MPECLib subset (89 problems).

5.2.2. Results by Problem Size Class

Table 3 stratifies outcomes by problem size. Small problems (typically 1–10 complementarity pairs) are entirely solved with a 100% convergence rate; all 29 achieve B-stationarity. Medium-sized problems also achieve a 100% convergence rate, with 36 of 47 instances certified as B-stationary and 11 as C-stationary. The timing results reveal that small problems solve rapidly (median 0.7 s), while medium problems require more substantial computation (median 80.1 s). Large problems ($n_x \geq 500$)

present the greatest challenge: of 13 instances, 6 converge (5 to B-stationarity and 1 to C-stationarity), while 4 timeout and 3 terminate with other failures (2 stationarity-unverifiable, 1 complementarity-infeasible). The non-converging cases are concentrated in the `tinque` and `frictional_block` families, both involving more than 1,000 complementarity pairs in their hardest instances.

Table 3. MPECLib outcomes stratified by problem size class (89 supported problems).

Size class	Total	B-stat.	C-stat.	Timeout	Other	Median wall-clock (s)
Small ($n_x < 50$)	29	29	0	0	0	0.7
Medium ($50 \leq n_x < 500$)	47	36	11	0	0	80.1
Large ($n_x \geq 500$)	13	5	1	4	3	232.3
Total	89	70	12	4	3	36.3

5.2.3. Results by Problem Family

Several observations stand out from the per-family results. *Complete success* (all instances solved to B-stationarity): `bard` (2/2), `bartruss` (6/6), `dempe` (2/2), `desilva` (1/1), `ex9` (4/4), `find_c` (12/12), `fjq` (1/1), `gauvin` (1/1), `hq` (1/1), `kehoe` (3/3), `kojshin` (2/2), `mss` (1/1), `nappi` (4/4), `outrata` (4/4), `oz` (1/1), and `tinloi` (1/1). *Mixed but mostly successful families:* the `find_a` family yields 9 B-stationary and 3 C-stationary solutions, while `find_b` yields 10 B-stationary and 2 C-stationary solutions. *C-stationarity without B-stationarity certificate*: The `aampec` family (6 instances, all with 69 complementarity pairs) converges to C-stationary points on all instances; the high complementarity density ($q/n \approx 0.96$) presents challenges for B-stationarity certification. *Non-converging cases*: The `tinque` family (6 large-scale instances with 3,000–5,700 variables from contact-mechanics discretisations) has 4 timeouts and 2 converged cases (1 B-stationary and 1 C-stationary). The `frictional_block` family (6 instances) has 2 stationarity-unverifiable terminations, 1 complementarity-infeasible termination, and 3 converged instances; these are known computationally intensive problems whose cost is dominated by sparse linear algebra.

5.2.4. Discussion of Non-Converging Cases and C-Stationarity

The `tinque` instances are large-scale problems (4,000–5,700 variables, 2,900–4,500 complementarity pairs) arising from contact-mechanics discretisations. Four of the

six supported instances exceed the 3600 s time limit, while the remaining two converge. These problems represent the computational frontier for regularisation-based MPEC solvers using open-source linear algebra; stronger sparse-direct or parallel sparse-factorisation backends would likely help on this family.

The `frictionalblock` instances range from 682 to 2,855 variables with 332 to 1,380 complementarity pairs. Two instances terminate with stationarity unverifiable and one is classified as complementarity-infeasible, while the remaining three converge to B-stationary points. The non-converged outcomes indicate numerical difficulty in the final stationarity and feasibility certification steps for this family.

The six `aampec` problems have $n_x = 72$ variables and $q = 69$ complementarity pairs, giving an extremely high complementarity density ($q/n \approx 0.96$). All six converge to C-stationarity. The high complementarity density makes B-stationarity certification particularly challenging, as the biactive set tends to be large at feasible points.

The 92.1% convergence rate [85.4% B-stationary among converged] on the 89-problem supported MPECLib subset demonstrate the effectiveness of MPECSS across a diverse problem collection. The 7 non-converging cases are concentrated in very large-scale problems (>1,000 complementarity pairs) that push the limits of the open-source MUMPS solver. The 12 C-stationary solutions, predominantly from `aampec` and the `find_*` families, represent problems where the LPEC post-processing correctly identifies that stronger stationarity certificates cannot be guaranteed.

5.3. MacMPEC Benchmark Results

MacMPEC [29] is an early and widely used MPEC benchmark collection, comprising 191 problems drawn from bilevel optimisation, economic equilibrium modelling, engineering design, and game theory. We use the 191-problem version as distributed on the AMPL/MacMPEC repository, which includes problems added after the original 2000 release. All experiments follow the same methodology described in Section 5.1 with the parameter settings in Table 1.

5.3.1. Summary Statistics

Table 4 summarises the overall outcomes. MPECSS converged on 184 of 191 problems (96.3% [87.5% B-stationary among converged]), certifying B-stationarity on 161 instances (84.3%) and C-stationarity on the remaining 23 (12.0%). Six problems (3.1%) reached the 3600 s timeout limit, and one problem (`bem-milanc30-s`) was classified as complementarity-infeasible.

Table 4. MPECSS outcome summary on the MacMPEC benchmark (191 problems, 3600 s time limit per problem).

Outcome	Count	Percentage
Converged – B-stationary	161	84.3%
Converged – C-stationary	23	12.0%
Total converged	*184*	*96.3%*
Timeout (>3600 s)	6	3.1%
Complementarity infeasible	1	0.5%
Total	**191**	**100%**

MPECSS demonstrates strong performance across all problem size categories. All 96 small problems ($n_x < 50$) converge successfully (100%), with 82 achieving B-stationarity. For the 71 medium-scale instances ($50 \leq n_x < 500$), 70 converge (98.6%), with 62 certified as B-stationary and 8 as C-stationary. For the 24 large-scale problems ($n_x \geq 500$), 18 converge successfully (75.0%), with 17 achieving B-stationarity and 1 reaching C-stationarity. The 7 non-converging cases include 6 timeouts from the `siouxfls`, `incid-set`, and `qpec` families and one complementarity-infeasible instance.

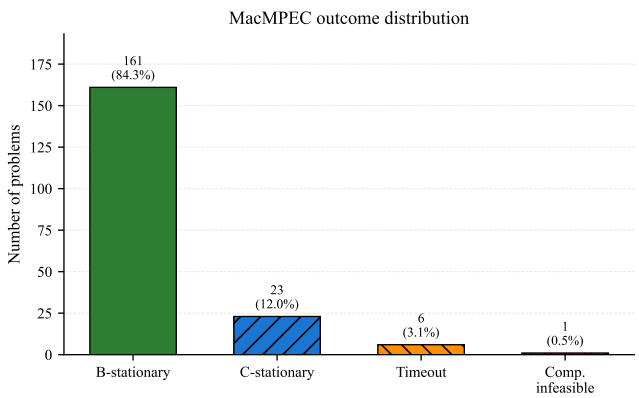

Figure 2. MPECSS outcome distribution on MacMPEC.

5.3.2. Results by Problem Size Class

Table 5 stratifies outcomes by problem size. Small problems ($n_x < 50$, typically 1–20 complementarity pairs) achieve a perfect 100% convergence rate across all 96 instances, with 82 achieving B-stationarity and 14 achieving C-stationarity. Medium-sized problems ($50 \leq n_x < 500$) achieve 98.6% convergence across 71 instances, with 62 instances (87.3%) achieving B-stationarity and 8 (11.3%) achieving C-stationarity. The wall-clock timing results reveal that small problems solve rapidly (median 0.8 s), while medium problems require moderate computation (median 7.5 s). Large problems ($n_x \geq 500$) present the greatest challenge: of 24 instances, 18 converge (75.0%), with 17 achieving B-stationarity and 1 achieving C-stationarity. The 6 timeout cases are concentrated in the `siouxfls`, `incid-set`, and `qpec` families, while `bem-milanc30-s` is the lone complementarity-infeasible instance.

Table 5. MacMPEC outcomes stratified by problem size class.

Size class	Total	B-stat.	C-stat.	Timeout	Other	Median wall-clock (s)
Small ($n_x < 50$)	96	82	14	0	0	0.8
Medium ($50 \leq n_x < 500$)	71	62	8	1	0	7.5
Large ($n_x \geq 500$)	24	17	1	5	1	781.4
Total	191	161	23	6	1	3.2

5.3.3. Results by Problem Family

Several observations stand out from the per-family results. Because the archived CSV stores the family field as the generic label `macmpec`, the family counts in this paragraph are reconstructed from the standard filename prefixes used in the MacMPEC distribution. *Near-complete B-stationarity* is observed for `pack` (44/45 B-stat, 1 C-stat), `gnash` (17/20 B-stat, 3 C-stat), `ex` (17/19 B-stat, 2 C-stat), `bard` (5/6 B-stat, 1 C-stat), and `design` (5/6 B-stat, 1 C-stat). *C-stationarity without a strong certificate:* the `qpec` family (10 instances) converges on 9 instances, with only 1 achieving B-stationarity and 8 achieving C-stationarity; the `bilevel` family (5 instances) converges on all 5, with 3 B-stationary and 2 C-stationary outcomes. *Timeout cases*: the `siouxfls` family (2 large-scale instances with 2,403 variables and 1,748 complementarity pairs) times out on both instances. The `incid-set` family contributes three additional timeout cases, and one `qpec` instance also times out. The named pair `ralph1` and `ralph2` both

converge to B-stationary points, while `ralphmod` is a separate converged C-stationary instance.

5.3.4. Discussion of Timeout and Failure Cases

The six timeout problems include: two `siouxfls` traffic network instances (`siouxfls` and `siouxfls1`) with 2,403 variables and 1,748 complementarity pairs each, three `incid-set` instances (`incid-set1-32`, `incid-set-32`, and `incid-set2c-32`) with up to 1,989 variables and 961 complementarity pairs, and one `qpec` instance (`qpec-200-4`) with 240 variables and 200 complementarity pairs. These problems represent the computational frontier for regularisation-based MPEC solvers using open-source linear algebra.

The single complementarity-infeasible case (`bem-milanc30-s`) is a large boundary element method problem with 3,436 variables and 1,464 complementarity pairs. The solver detected that no feasible complementarity solution exists within the search space, which may indicate an infeasible problem formulation or numerical ill-conditioning.

The 96.3% convergence rate [87.5% B-stationary among converged] on MacMPEC demonstrates the robustness of MPECSS across diverse problem origins. The 7 non-converging cases are concentrated in large-scale problems that push the limits of the open-source MUMPS linear solver.

MPECopt [35] is a recent MPEC solver with global convergence guarantees to B-stationary points. A detailed comparison on the MacMPEC collection is provided in Section 5.5. The comparison focuses on (i) convergence rates, (ii) B-stationarity certification rates, and (iii) wall-clock time. Note that MPECopt requires MATLAB and Gurobi, whereas MPECSS is fully open-source.

5.4. NOSBENCH Benchmark Results

NOSBENCH [36] is a recently introduced benchmark collection comprising 603 problems arising from optimal control of nonsmooth dynamical systems. These problems are generated by discretising piecewise-affine (PWA) and piecewise-smooth (PWS) op-

timal control problems, yielding MPECs with a distinctive complementarity structure: the complementarity pairs arise from the switching conditions in the dynamics rather than from equilibrium constraints. Problem sizes range from small (under 50 variables) to medium-scale (several hundred variables), with complementarity counts typically proportional to the discretisation horizon. All experiments follow the same methodology described in Section 5.1 with the parameter settings in Table 1. The repository contains four archived official NOSBENCH group CSVs together with their matching run-environment JSON files; however, the current archived bundle still reflects a benchmark-export race between synthetic timeout/crash bookkeeping and late worker results. As a consequence, the checked-in CSV bundle is retained for auditability but not yet frozen as the publication source of truth for NOSBENCH headline counts.

5.4.1. Summary Statistics

Table 6 records the archival status of the current checked-in NOSBENCH bundle. Across the four official group CSVs, the repository contains 610 terminal rows for 603 unique problem names. The corresponding run-environment JSON files also expose the same mismatch through their `problem_count` and `result_count` fields (151/153, 151/153, 151/152, and 150/152 for groups 1–4). Manual inspection shows that the duplicated rows are concentrated in seven problems and arise from timeout/crash or timeout/late-result pairs. Because these duplicated terminal rows would contaminate any publication table, the NOSBENCH manuscript numbers are left as placeholders until the full suite is regenerated from the corrected exporter.

Table 6. Archival status of the checked-in NOSBENCH official bundle.

Item	Value
Official group CSV files	4
Unique problem names across the bundle	603
Continued [to be completed]	

The duplicate-row issue is now fixed in the benchmark exporter in the current code base; the remaining task for NOSBENCH is therefore a clean full-suite refresh rather than an algorithmic redesign. Until that refresh is completed, we keep the NOSBENCH section as an explicit publication placeholder instead of reporting stale percentages

from an older manuscript draft.

5.4.2. Results by Problem Size Class

Table 7 is intentionally left as a manuscript placeholder. The size-stratified publication table will be regenerated from the corrected archive so that each NOSBENCH problem contributes a single terminal row before the counts are broken down by `problem_size_mode`.

Table 7. NOSBENCH size-stratified publication placeholder.

Item	Status
Small / medium / large outcome counts	to be regenerated from corrected CSVs
Median wall-clock summaries	to be regenerated from corrected CSVs
Reason for placeholder	current archive contains duplicate timeout/crash rows

5.4.3. Results by Problem Family

The family-level publication discussion is likewise postponed until the corrected archive is available. The current checked-in bundle is still useful diagnostically: the duplicated rows are concentrated in a small set of families (`3CPCLS`, `CARHYS`, and `DAOBCLS`), which is consistent with the timeout/crash export race identified in the benchmark harness.

5.4.4. Discussion of Timeout and Failure Cases

The duplicated examples in the archived CSVs are all failure-adjacent: timeout/crash pairs for four `3CPCLS` and `CARHYS` instances, plus a timeout/late-result pair for one `DAOBCLS` instance. This is precisely the class of ambiguity that the exporter fix in the current revision removes. Once the NOSBENCH suite is rerun or the archived bundle is re-exported from the corrected harness, the summary, size-stratified, and family-level tables in this section should be regenerated directly from that corrected bundle before final publication.

5.5. Performance Profile Analysis

Performance profiles, introduced by Dolan and Moré [10], provide a standardised methodology for comparing optimisation solvers across a benchmark suite. For each

problem p and solver s, let $t_{p,s}$ denote the performance metric (e.g., complementarity residual or wall-clock time). The performance ratio is defined as

$$r_{p,s} = \frac{t_{p,s}}{\min_s t_{p,s}},$$

and the performance profile $\rho_s(\tau)$ gives the fraction of problems for which solver s achieves a performance ratio at most τ:

$$\rho_s(\tau) = \frac{|\{p : r_{p,s} \leq \tau\}|}{|\mathcal{P}|},$$

where $\mathcal{P}$ is the set of all benchmark problems. A solver with a higher $\rho_s(\tau)$ for all τ is uniformly better.

A direct MPECSS–MPECopt comparison on MacMPEC will be inserted once the curated side-by-side result bundle and the corresponding profile figure are frozen. The intended comparison metric is the final complementarity residual r_{comp}, with a problem counted as "solved" only when $r_{\text{comp}} \leq 10^{-6}$ and the final certificate is B-stationarity; unsolved problems are assigned $r_{p,s} = \infty$.

5.6. Performance and Benchmark Comparison

Table 8 summarizes overall performance across the three benchmark suites.

Table 8. Frozen archived benchmark summary for the current manuscript bundle.

Benchmark	**Total Probs**	**Converged** (%)	**Convergence Rate**	**Avg wall-clock Time (s)**	**Avg Compl. Viol.**	**B-stat Verified (%)**
MacMPEC	191	184	96.3%	166.5	2.2×10^{-8}	87.5%
MPECLib	89	82	92.1%	253.1	4.1×10^{-7}	85.4%
NOSBENCH	603	TBD	TBD	TBD	TBD	TBD
Frozen subtotal	280	266	95.0%	193.2	1.4×10^{-7}	86.8%

Cross-suite difficulty stratification is deferred to the final archived benchmark bundle, where the size-class definitions and family metadata will be regenerated from a single machine-produced summary rather than manually assembled across suites. In the present manuscript revision, MacMPEC and MPECLib are the two frozen suites, while NOSBENCH remains a placeholder pending corrected full-suite regeneration.

6. Conclusions

In this paper, we presented MPECSS in a fully open-source environment and reported results on three benchmark suites. The outcomes are encouraging and show that the proposed approach performs robustly across the tested problems.

Future work will compare MPECSS with commercial solvers on the same benchmark suites and assess the doing comparsion with our reported results. The main limitation observed in the current study is the wall-clock time required for difficult instances. A promising direction is to explore more advanced efficient and GPU-accelerated linear-algebra libraries to reduce solve times and improve scalability.

Acknowledgements

The authors thank the developers of CasADi and IPOPT for providing powerful open-source tools that enabled this research and the broader optimization community. The authors thank Anton Pozharskiy for translating the MacMPEC test collection to the CasADi format used in the numerical experiments. First author Saurabh want to specifically ackwolege the mentorship and support of their supervisor Dr. Kunwar Vijay Kumar Signh for providing guidance and significant intellectual contributions to the development of the MPECSS algorithm and the design of the numerical experiments.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used the following AI-assisted tools: Claude Opus 4 (Anthropic) and ChatGPT-5 (OpenAI) for brainstorming, deep technical writing assistance, and exploration of algorithmic ideas; Claude Sonnet 4 (Anthropic) for drafting and editing manuscript sections; Claude Haiku 4.5 (Anthropic) for literature review and language improvement; Gemini 3.1 Pro (Preview) (Google) for

additional literature review and language checking; and GitHub Copilot (Microsoft, powered by OpenAI) for code development assistance. After using these tools, the authors reviewed and edited all generated content and take full responsibility for all aspects of this publication, including the accuracy of mathematical derivations, the authenticity of reported results, and compliance with publication ethics. These AI tools did not make significant intellectual contributions to the research, nor did they determine any aspect of the experimental outcomes.

Funding

The authors report no funding.

Data availability statement

The benchmark problem instances used in this study (MacMPEC, MPECLib, and NOSBENCH) are publicly available as referenced in the text. The checked-in repository contains the archived benchmark CSV files, available run-environment metadata under `results/outputs_official_*`, and the scripts used to generate the reported figures and tables. These materials are available at `https://github.com/mpecssalgorithm/MPECSS`.

Software availability

The MPECSS algorithm is freely available under the Apache 2.0 licence. The source code, benchmark datasets, and scripts required to reproduce all reported results are available on GitHub at `https://github.com/mpecssalgorithm/MPECSS`. The archived official MacMPEC and MPECLib benchmark artifacts discussed in this manuscript were produced at git commit `33adefe` under Python 3.12.12 with CasADi 3.7.2, NumPy 2.0.2, pandas 2.3.3, SciPy 1.16.3, psutil 5.9.5, and matplotlib 3.10.0. The accompanying run-environment JSON records a Linux execution environment on an Intel Xeon or AMD EPYC platform with 31.35 GB RAM; the manuscript's numerical discussion refers to those archived run-environment files where

applicable. The repository also contains a NOSBENCH official bundle at the same commit; because that bundle is being refreshed after correcting duplicate timeout/crash CSV rows, the NOSBENCH manuscript tables are marked as pending in the current revision.

References

[1] J.A.E. Andersson, J. Gillis, G. Horn, J.B. Rawlings, and M. Diehl, *CasADi: A software framework for nonlinear optimization and optimal control*, Math. Program. Comput. 11(1) (2019), pp. 1–36.

[2] M. Anitescu, *On using the elastic mode in nonlinear programming approaches to mathematical programs with complementarity constraints*, SIAM J. Optim. 15(4) (2005), pp. 1203–1236.

[3] P.R. Amestoy, I.S. Duff, J.-Y. L'Excellent, and J. Koster, *A fully asynchronous multifrontal solver using distributed dynamic scheduling*, SIAM J. Matrix Anal. Appl. 23(1) (2001), pp. 15–41.

[4] B.T. Baumrucker, J.G. Renfro, and L.T. Biegler, *MPEC problem formulations and solution strategies with chemical engineering applications*, Comput. Chem. Eng. 32(12) (2008), pp. 2903–2913.

[5] B. Colson, P. Marcotte, and G. Savard, *An overview of bilevel optimization*, Ann. Oper. Res. 153(1) (2007), pp. 235–256.

[6] A.R. Conn, N.I.M. Gould, and Ph.L. Toint, *Trust-Region Methods*, MPS-SIAM Series on Optimization, SIAM, Philadelphia, 2000.

[7] V. DeMiguel, M.P. Friedlander, F.J. Nogales, and S. Scholtes, *A two-sided relaxation scheme for mathematical programs with equilibrium constraints*, SIAM J. Optim. 16(2) (2005), pp. 587–609.

[8] S. Dempe, *Foundations of Bilevel Programming*, Kluwer Academic Publishers, Dordrecht, 2002.

[9] S. Dirkse and M. Bussieck, *MPECLib: A collection of mathematical programs with equilibrium constraints*, GAMS World test library, available via `https://www.gams.com/modlib/libhtml/index.html` and mirrored at `https://github.com/GAMS-dev/gamsworld` (accessed March 2026).

[10] E.D. Dolan and J.J. Moré, *Benchmarking optimization software with performance profiles*,

Math. Program. 91(2) (2002), pp. 201–213.

[11] F. Facchinei and J.-S. Pang, *Finite-Dimensional Variational Inequalities and Complementarity Problems*, Springer Series in Operations Research, Springer, New York, 2003.

[12] H.J. Ferreau, C. Kirches, A. Potschka, H.G. Bock, and M. Diehl, *qpOASES: A parametric active-set algorithm for quadratic programming*, Math. Program. Comput. 6(4) (2014), pp. 327–363.

[13] M.C. Ferris and J.-S. Pang, *Engineering and economic applications of complementarity problems*, SIAM Rev. 39(4) (1997), pp. 669–713.

[14] A. Fischer, *A special Newton-type optimization method*, Optimization 24(3–4) (1992), pp. 269–284.

[15] M.L. Flegel and C. Kanzow, *On M-stationary points for mathematical programs with equilibrium constraints*, J. Math. Anal. Appl. 310(1) (2005), pp. 286–302.

[16] R. Fletcher and S. Leyffer, *Solving mathematical programs with complementarity constraints as nonlinear programs*, Optim. Methods Softw. 19(1) (2004), pp. 15–40.

[17] R. Fletcher, S. Leyffer, D. Ralph, and S. Scholtes, *Local convergence of SQP methods for mathematical programs with equilibrium constraints*, SIAM J. Optim. 17(1) (2006), pp. 259–286.

[18] P.E. Gill, W. Murray, and M.A. Saunders, *SNOPT: An SQP algorithm for large-scale constrained optimization*, SIAM Rev. 47(1) (2005), pp. 99–131.

[19] T. Hoheisel, C. Kanzow, and A. Schwartz, *Theoretical and numerical comparison of relaxation methods for mathematical programs with complementarity constraints*, Math. Program. 137(1–2) (2013), pp. 257–288.

[20] X.M. Hu and D. Ralph, *Convergence of a penalty method for mathematical programming with complementarity constraints*, J. Optim. Theory Appl. 123(2) (2004), pp. 365–390.

[21] X. Hu and D. Ralph, *Using EPECs to model bilevel games in restructured electricity markets with locational prices*, Oper. Res. 55(4) (2007), pp. 809–824.

[22] Q. Huangfu and J.A.J. Hall, *Parallelizing the dual revised simplex method*, Math. Program. Comput. 10(1) (2018), pp. 119–142.

[23] A. Kadrani, J.P. Dussault, and A. Benchakroun, *A new regularization scheme for mathematical programs with complementarity constraints*, SIAM J. Optim. 20(1) (2009), pp. 78–103.

[24] C. Kanzow and A. Schwartz, *A new regularization method for mathematical programs with complementarity constraints with strong convergence properties*, SIAM J. Optim.

23(2) (2013), pp. 770–798.

[25] C. Kanzow and A. Schwartz, *The price of inexactness: convergence properties of relaxation methods for mathematical programs with complementarity constraints revisited*, Math. Oper. Res. 40(2) (2015), pp. 253–275.

[26] C. Kanzow, *Some noninterior continuation methods for linear complementarity problems*, SIAM J. Matrix Anal. Appl. 17(4) (1996), pp. 851–868.

[27] A. Klarbring and G. Björkman, *A mathematical programming approach to contact problems with friction and varying contact surface*, Comput. Struct. 30(5) (1988), pp. 1185–1198.

[28] S. Leyffer, G. López-Calva, and J. Nocedal, *Interior methods for mathematical programs with complementarity constraints*, SIAM J. Optim. 17(1) (2006), pp. 52–77.

[29] S. Leyffer, *MacMPEC: A collection of mathematical programs with equilibrium constraints*, Technical Report ANL/MCS-P817-0500, Argonne National Laboratory, 2000.

[30] S. Leyffer and T. Munson, *Solving multi-leader-common-follower games*, Optim. Methods Softw. 25(4) (2010), pp. 601–623.

[31] G.-H. Lin and M. Fukushima, *A modified relaxation scheme for mathematical programs with complementarity constraints*, Ann. Oper. Res. 133(1–4) (2005), pp. 63–84.

[32] Z.-Q. Luo, J.-S. Pang, and D. Ralph, *Mathematical Programs with Equilibrium Constraints*, Cambridge University Press, Cambridge, 1996.

[33] Q. Meng, H. Yang, and M.G.H. Bell, *An equivalent continuously differentiable model and a locally convergent algorithm for the continuous network design problem*, Transp. Res. Part B Methodol. 35(1) (2001), pp. 83–105.

[34] J. Nocedal and S.J. Wright, *Numerical Optimization*, 2nd ed., Springer Series in Operations Research and Financial Engineering, Springer, New York, 2006.

[35] A. Nurkanović and S. Leyffer, *A globally convergent method for computing B-stationary points of mathematical programs with equilibrium constraints*, arXiv:2501.13835v4, 2025.

[36] A. Nurkanović, A. Pozharskiy, and M. Diehl, *Solving mathematical programs with complementarity constraints arising in nonsmooth optimal control*, Vietnam J. Math. (2024), pp. 1–39.

[37] J.V. Outrata, M. Kočvara, and J. Zowe, *Nonsmooth Approach to Optimization Problems with Equilibrium Constraints: Theory, Applications and Numerical Results*, Kluwer Academic Publishers, Dordrecht, 1998.

[38] A.U. Raghunathan and L.T. Biegler, *An interior point method for mathematical programs*

with complementarity constraints (MPCCs), SIAM J. Optim. 15(3) (2005), pp. 720–750.

[39] D. Ralph and S.J. Wright, *Some properties of regularization and penalization schemes for MPECs*, Optim. Methods Softw. 19(5) (2004), pp. 527–556.

[40] R.T. Rockafellar and R.J.-B. Wets, *Variational Analysis*, Springer, Berlin, 1998.

[41] H. Scheel and S. Scholtes, *Mathematical programs with complementarity constraints: stationarity, optimality, and sensitivity*, Math. Oper. Res. 25(1) (2000), pp. 1–22.

[42] S. Scholtes, *Convergence properties of a regularization scheme for mathematical programs with complementarity constraints*, SIAM J. Optim. 11(4) (2001), pp. 918–936.

[43] S. Steffensen and M. Ulbrich, *A new relaxation scheme for mathematical programs with equilibrium constraints*, SIAM J. Optim. 20(5) (2010), pp. 2504–2539.

[44] A. Wächter and L.T. Biegler, *On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming*, Math. Program. 106(1) (2006), pp. 25–57.

[45] J.J. Ye, *Necessary and sufficient optimality conditions for mathematical programs with equilibrium constraints*, J. Math. Anal. Appl. 307(1) (2005), pp. 350–369.